

Multi-Agent Communication Transformer with Leader-Follower Architecture

Chao Wang¹, Xiang Xu^{2*}, Ziyang Ouyang^{2†}

^{1,2*}Southern University of Science and Technology, Shenzhen, China.

²Beijing Jiaotong University, Beijing, China.

*Corresponding author(s). E-mail(s): xux3@sustech.edu.cn;

Contributing authors: 12332665@mail.sustech.edu.cn;

24125704@bjtu.edu.cn;

†These authors contributed equally to this work.

Abstract

In the realm of multi-agent reinforcement learning (MARL), communication is of utmost importance for agents to cooperate effectively. However, current communication methods encounter difficulties in extracting effective information under strict independence and limited communication conditions. This paper presents a novel approach named LF-Commer with a leader-follower architecture and multi-agent Transformer. It utilizes the clustering algorithm to divide regions based on agent distribution and an incremental update strategy to handle dynamic position changes. The introduced Transformer architecture features an encoder learning joint observation representations with communication graph and a decoder outputting actions auto-regressively. Notably, the communication trust score is proposed to optimize the attention mechanism, reducing communication redundancy. In benchmarks such as the StarCraftII Multi-Agent Challenge (SMAC) and the Multi-Agent Particle Environment (MPE), LF-Commer demonstrates outstanding performance with high success rate and cumulative reward. It significantly improves cooperation efficiency and algorithm stability, providing an effective solution for communication and cooperation in MARL.

Keywords: Multi-Agent Reinforcement Learning, Transformer, Leader-Follower, Multi-Agent Communication

1 Introduction

Multi-Agent Reinforcement Learning (MARL) has emerged in response to the growing need for models that can handle cooperative and collaborative scenarios in real-world environments[1]. These scenarios often involve multiple decision-making agents, each making decisions within a multi-agent system. As an extension of reinforcement learning, MARL applies the basic paradigm of decision optimization to a multi-agent system, allowing agents to interact, communicate, and cooperate to complete joint tasks or solve complex problems[2]. After receiving their individual observations o_i , each agent decides on an action a_i through its decision-making module[3]. These actions are then combined to form a joint action $a_{\text{tot}} = \langle a_1, a_2, \dots, a_n \rangle$, which is applied to the environment to collectively receive the reward and the next set of observations o'_i . As an important research direction within distributed artificial intelligence, MARL has significant practical applications in fields such as industrial manufacturing[4], transportation[5], unmanned swarm systems[6], and military decision-making[7].

Although multi-agent coordination technology has seen considerable progress in the past decade, challenges remain due to the complexity of interactions between agent-agent and agent-environment[8]. In many real-world scenarios, a centralized controller is impractical, as agents often operate independently without synchronization[9]. Centralized training with decentralized execution (CTDE)[10] offers a compromise between centralized and decentralized approaches, but does not always outperform purely decentralized training. Additionally, the vast joint action space in MARL can lead to the *curse of dimensionality*, and individual updates to agents' policies may cause instability in the environment[11].

In multi-agent system (MAS), integrating training information from all agents poses significant challenges in practical applications. As a result, communication becomes a critical and efficient mechanism. Communication-based multi-agent algorithms[12–15] aim to enable agents to sense and process information from others, allowing communication to compensate for missing environmental information and facilitating better cooperation among agents. However, under stringent requirements for full independence and limited information exchange, extracting effective communication information while maintaining fully independent learning remains a significant challenge[16].

To address the challenges inherent in MAS communication, we introduce leveraging the serialization properties of Transformer to process the naturally sequential characteristics of multi-agent system[17]. Transformers are highly effective at modeling sequential data and capturing complex dependencies, making them well-suited for processing sequence-like interactions in MAS. Sequence models (SMs) have demonstrated exceptional capabilities in recent years, particularly in natural language processing (NLP)[18], where they excel at capturing sequential characteristics and complex dependencies in data. Models such as GPT[19] and BERT[20], leveraging auto-regressive or bidirectional Transformer architectures, have achieved remarkable success in modeling the sequential nature of language. As the versatility of Transformer continues to emerge, the principles underlying sequence models have been increasingly extended to other domains, including computer vision (CV)[21] and multi-model

tasks[22]. MAT[17] represents an innovative application of sequence models in the context of MAS. By treating multiple agents and their interactions within MAS as a temporal sequence, MAT employs the self-attention mechanism of Transformer to capture intricate inter-agent dependencies and dynamic interactions. In order to take advantage of the excellent properties of transformer in multi-agent, we propose the **LF-Commer** algorithm. Our contributions can be summarized as follows:

- LF-Commer introduces a leader-follower communication framework that optimizes communication topology through dynamic clustering. By grouping agents based on spatial distribution and designating leaders, it minimizes redundant communication and enables more streamlined and efficient exchange of information.
- Our approach harnesses the sequential modeling power of Transformers to effectively capture temporal interactions within multi-agent systems. This architecture enables efficient encoding of agents’ observations and communication data. Furthermore, the integration of an auto-regressive approach for parallel computation of agent actions significantly accelerates the training process.
- Extensive evaluations on several challenging MARL benchmarks demonstrate that LF-Commer consistently achieves superior performance, delivering results aligned with intuitive expectations. Across diverse cooperative tasks, our method outperforms robust baseline approaches, highlighting its effectiveness and adaptability.

2 Related Work

2.1 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) has gained attention for tackling complex problems in Markov games[1], particularly in online environments where agents collaborate towards a shared reward. These approaches are categorized into centralized training[23] and decentralized execution[24]. Centralized training learns a single policy for joint actions, while decentralized execution allows each agent to optimize its own reward function. Methods like MADDPG[25] and COMA[26] utilize centralized critics to handle local and shared rewards, respectively. To mitigate non-stationarity in MARL, centralized training with decentralized execution (CTDE)[10] combines the advantages of both. Our work proposes an innovative CTDE-based approach that enhances local information exchange between agents to accelerate policy iteration, aiming to improve cooperation and algorithm stability.

2.2 Multi-Agent Communication

Communication among agents is prominent for coordinated actions in multi-agent system. Existing research in this domain focuses on how to extract valuable information effectively. For instance, ATOC[27] framework employs a gate mechanism to determine when agents should communicate with others. I2C[14] facilitates interactions with relevant and influential agents identified by causal inference. Despite these advances, most approaches[15, 28, 29] emphasize extracting useful information from partial observations of current or past states. In contrast, approaches such as TarMAC[15] and ToM2C[30] focus on dynamic communication, allowing agents to selectively exchange

information. Such nonstatic communication topologies allow for the dynamic capture of critical inter-agent information, transcending the constraints of fixed communication architectures[12]. This adaptability is particularly advantageous in environments where the positions and relationships of the agents change dynamically.

2.3 Leader-Follower Framework

Leader-follower framework has found extensive application in multi-agent systems [31, 32], particularly within the control domain [33]. In this framework, one or more agents are designated as leaders, while the remaining agents act as followers, responding to the actions of the leaders. This paradigm has demonstrated superior performance in the handling of complex cooperative tasks. However, challenges remain in determining the selection of leaders and ensuring the robustness and efficiency of the system [32]. In this work, we adopt a specialized approach to designate certain agents as leaders and establish dedicated communication channels between leaders and followers. Within these channels, all agents share local observations and actions, facilitating enhanced coordination and information exchange across the system.

3 Methodology

The primary objective of our proposed method is to enhance communication within a leader-follower framework to tackle the challenges associated with multi-agent collaboration. This work is particularly dedicated to learning the dynamic adjustment of communication topologies to fit variations in agent positions and to the design of efficient communication channels for the exchange of observational data among agents. Through the implementation of these mechanisms, our approach seeks to enhance both coordination and adaptability in environments characterized by dynamism.

3.1 Problem Formulation

Cooperative multi-agent tasks can be modeled as decentralized partially observable Markov decision processes (Dec-POMDPs)[11], defined as $\mathbb{G} = \langle S, A, P, r, O, n, \gamma \rangle$. In this framework, S represents the global state space, while A denotes the joint action space formed by the product of the action spaces of individual agents. At each time step t , agents choose actions $a \in A$, resulting in a joint action that drives a state transition $s \rightarrow s'$, determined by the transition probability function $P(s'|s, a) : S \times A \times S \rightarrow [0, 1]$. Agents share a common reward function $r(s, a) : S \times A \rightarrow \mathbb{R}$, and $\gamma \in [0, 1]$ serves as the discount factor. In partially observable environments, each agent receives a local observation $o \in O$, derived from the global state through the observation function $O(s, a) : S \times A \rightarrow S'$. Agents rely on their action-observation histories to condition a stochastic policy π_t , which generates a joint action-value function $Q^\pi(s^t, a^t) = \mathbb{E}_{s^{t+1:\infty}, a^{t+1:\infty}} [R^t | s^t, a^t]$, where $R^t = \sum_{i=0}^{\infty} \gamma^i r^{t+i}$ represents the discounted cumulative reward.

3.2 leader-follower framework application

To implement the leader-follower framework effectively, agents are first partitioned into multiple sub-regions based on spatial distribution. For this purpose, we employ the DBSCAN algorithm[34], which clusters agents based on density. DBSCAN defines two critical parameters: ϵ (neighborhood radius) and minPts (minimum points). For a given agent p , its communication radius can be defined as ϵ . The ϵ -neighborhood is then expressed as:

$$N_\epsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}, \quad (1)$$

where $\text{dist}(p, q)$ represents the distance between agents p and q , and D is the set of all agent positions. If $|N_\epsilon(p)| \geq \text{minPts}$, agent p is identified as the central point, serving as the center of the subregion. All agents within $N_\epsilon(p)$ are assigned to the same subregion. If an agent does not satisfy the core point condition but resides within the ϵ neighborhood of a core point, it is classified as a boundary point. Otherwise, it is treated as noise. This density-based clustering approach does not require a priori specification of the number of clusters, making it well suited for partitioning complex spatial distributions. Furthermore, it can identify clusters of arbitrary shapes while filtering out noise.

Given that agent positions may dynamically change, an incremental update strategy is applied at fixed time intervals. Specifically, when an agent's current position undergoes a significant change, it is first marked as p' . The ϵ -neighborhood $N_\epsilon(p')$ is then recalculated to determine whether cluster expansion is necessary. If new neighborhood $N_\epsilon(p')$ contains core points that are not yet assigned to cluster C_i , these points are added to the cluster. The expansion process is formally defined as

$$C_i = C_i \cup \{q \in N_\epsilon(p') \mid q \text{ is a core point and unassigned}\}, \quad (2)$$

where C_i denotes the i -th subregion containing all agents directly connected to the core point. This incremental update method effectively reduces unnecessary computation by focusing only on regions with significant changes, thereby improving the system's real-time performance.

Once subregions are identified via DBSCAN, the leader for each subregion is selected by its core point, coordinating task allocation and decision-making within the subregion. The leader is responsible for gathering observational information from its followers and making decisions based on the information to generate actions. Followers execute actions based on the leader's instruction, relying solely on local observations and lacking access to global information. Communication between a leader and its followers occurs via local communication channels to ensure coordinated task execution.

Within leader-follower framework, challenges such as communication delays and disruptions may arise. To address the problem, we define communication trust score c_{ij} to quantify the contribution of follower j to leader i in terms of communication reliability, $c_{ij} = \frac{\text{dist}(i,j)}{\tau}$, where τ represents the measurable time delay in communication. This metric enables a quantitative assessment of communication reliability, facilitating robust coordination. A simple leader-follower demo shown as Fig. 1

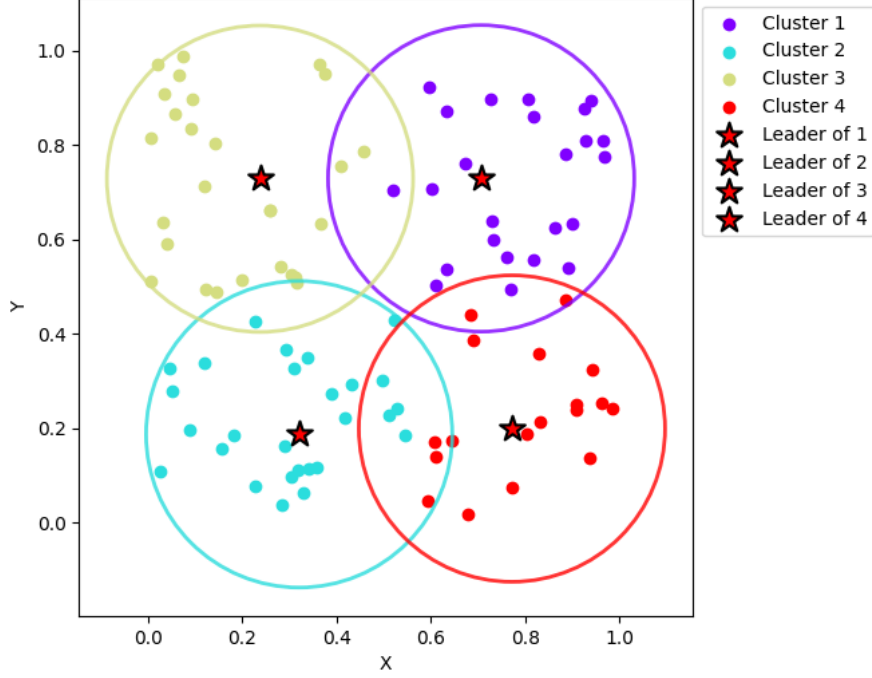


Fig. 1: This is a diagram of a leader-follower architecture with randomly distributed agents.

3.3 Transformer Architecture

To enable a sequential modeling paradigm for MARL, we introduce the Multi-Agent Transformer (MAT) [17]. The attention mechanism within the Transformer architecture enables batch processing of operations, leveraging $a_{i_0}^t, \dots, a_{i_{n-1}}^t$ in the buffer to predict $a_{i_1}^t, \dots, a_{i_n}^t$ while simultaneously training and updating policies. As depicted in Figure 2, the communication Transformer comprises an encoder and a decoder. The encoder is responsible for learning a representation of the joint observations, while the decoder operates in an auto-regressive manner to output actions for each agent. This design allows communication Transformer to effectively capture and model the dependencies among agents' actions and observations, ensuring robust decision-making in complex multi-agent system. The overall architecture graph is shown as Fig. 2.

The encoder, parameterized by ϕ , processes an unordered sequence of observations $o_1, \dots, o_n$ through a series of computational layers. Each layer incorporates a relation-aware mechanism combined with a multi-layer perceptron (MLP) and employs residual connections to address gradient vanishing and prevent performance degradation in deeper networks. Before this, node and edge embeddings are introduced to enrich input features before further computation [12]. In the Multi-Head Attention mechanism, the interaction between the leader observation o_i and follower observation o_j is quantified by the dot product of their respective query and key vectors:

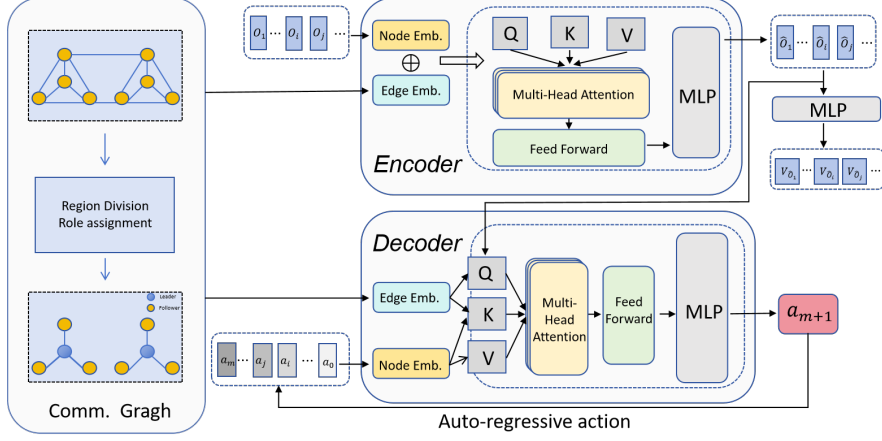


Fig. 2: Communication architecture comprises a communication graph for role assignment, an encoder with node and edge embeddings processing observations through multi-head attention and feed-forward networks, and a decoder predicting actions using similar mechanisms.

$$S_{ij} = f(o_i, o_j) = o_i W_q^T W_k o_j, \quad (3)$$

where W_q, W_k are learnable matrices. S_{ij} can be interpreted as implicit information associated with the edge $e_{j \rightarrow i}$, where leader i queries information sent by follower j . To reduce communication redundancy and efficiently utilize communication message within the leader-follower framework, we update S_{ij} as follows:

$$\begin{aligned} S_{ij} &= f'(o_i, o_j, e_{i \rightarrow j}, e_{j \rightarrow i}) \\ &= (o_i + c_{ij} \times e_{i \rightarrow j}) W_q^T W_k (o_j + c_{ji} \times e_{j \rightarrow i}), \end{aligned} \quad (4)$$

where c_{ij} is communication trust score. We set $c_{ji} = 1$ because communication between leader and follower must be sent by follower, while leader only sends action instruction to follower. Accordingly, it will be equitable to each follower. After encoding, the observation representations are denoted as $(\hat{\mathbf{o}}_1, \dots, \hat{\mathbf{o}}_i, \hat{\mathbf{o}}_j, \dots)$. These representations integrate both the raw observation information of the agents and the communication information derived from the leader-follower framework. In this work, we employ the encoder as the critic to evaluate the policy and optimize the system. The loss function for the encoder is defined as:

$$L_{\text{Encoder}}(\phi) = \frac{1}{Tn} \sum_{i=1}^n \sum_{t=0}^{T-1} [R(\mathbf{o}^t, \mathbf{a}^t) + \gamma V_{\bar{\phi}}(\hat{\mathbf{o}}_i^{t+1}) - V_{\phi}(\hat{\mathbf{o}}_i^t)]^2, \quad (5)$$

Here, $\bar{\phi}$ represents the parameters of the target network, which shares the same structure as the current critic network but updates at a slower pace. The target network is maintained by periodically copying the parameters ϕ of the current critic to $\bar{\phi}$, ensuring a stable target for training. This mechanism mitigates training instabilities by

reducing the correlations between the target and the current value estimates, a crucial technique in modern reinforcement learning algorithms[3].

The decoder, parameterized by θ , operates on the embedded joint actions $a_{0:m-1}$ for $m = 1, \dots, N$ using a series of decoding layers. Each layer employs a feature enhancement mechanism to compute attention between the encoded actions and explicit communication signals. Additionally, multi-layer perceptrons (MLPs) and residual connections are integrated to improve learning efficiency and stability. To encode communication constraints, an adjacency matrix mask is applied, ensuring attention is restricted to agent i and agents j with $j < i$. This restriction enables sequential updates, allowing the decoder to generate actions auto-regressively: $\pi_{\theta}^{m+1}(a_{m+1}|\hat{\mathbf{o}}_{1:n}, a_{0:m})$. The decoder is trained using the PPO algorithm [35]:

$$\begin{aligned} L_{\text{Decoder}}(\theta) &= -\frac{1}{Tn} \sum_{m=1}^n \sum_{t=0}^{T-1} \min \left(r_m^t(\theta) \hat{A}^t, \text{clip}(r_m^t(\theta), 1 \pm \epsilon) \hat{A}^t \right), \\ r_m^t(\theta) &= \frac{\pi_m^{\theta}(a_m^t | \hat{\mathbf{o}}_{1:n}^t, \hat{\mathbf{a}}_{1:m-1}^t)}{\pi_m^{\theta_{\text{old}}}(a_m^t | \hat{\mathbf{o}}_{1:n}^t, \hat{\mathbf{a}}_{1:m-1}^t)}, \end{aligned} \tag{6}$$

where the estimated joint advantage function is expressed as: $\hat{V}^t = \frac{1}{n} \sum_{i=1}^n V(\hat{o}_i^t)$. During inference, actions are produced auto-regressively, feeding a_m back into the decoder to generate a_{m+1} in sequence, starting from a_0 to a_{n-1} . In contrast, during training, all actions $a_{1:n}$ are computed in parallel, as $a_{1:n-1}$ are pre-collected in the replay buffer. By leveraging leader-follower communication, agents can efficiently select actions with lower communication costs compared to fully connected architectures, minimizing overhead. The full procedure is detailed in Algorithm 1.

Algorithm 1 Communication Transformer Based on Leader-Follower Architecture

```
1: Input: Batch size  $B$ , number of agents  $N$ , episodes  $K$ , steps per episode  $T$ .
2: Initialize: Encoder  $\{\phi\}$ , Decoder  $\{\theta\}$ , Replay buffer  $\mathcal{B}$ , leader-follower communication information and trust score matrix  $\psi, \mathbf{c} \in \mathbb{R}^{n \times n}$ .
3: for  $k = 0, 1, \dots, K - 1$  do
4:   for  $t = 0, 1, \dots, T - 1$  do
5:     Collect a sequence of observations  $(o_1^t, \dots, o_n^t)$  from environments.
6:     // The inference phase
7:     Generate the communication matrix  $\psi, \mathbf{c}$  by Equation 1 and Equation 2.
8:     Generate  $(\hat{o}_1^t, \dots, \hat{o}_n^t)$  by feeding  $(o_1, \dots, o_n)$  and  $\psi$  to Encoder.
9:     Input  $(\hat{o}_1^t, \dots, \hat{o}_n^t)$  and  $\psi, \mathbf{c}$  to Decoder.
10:    for  $m = 0, 1, \dots, n - 1$  do
11:      Infer  $\mathbf{a}_{m+1}^t$  via auto-regressive manner.
12:    end for
13:    Execute joint actions  $(a_0^t, \dots, a_n^t)$  and collect the joint reward  $R(o^t, a^t)$ .
14:    Insert  $(o^t, a^t, R(o^t, a^t))$  into  $\mathcal{B}$ .
15:  end for
16:  // The training phase
17:  Sample a random minibatch of  $B$  steps from  $\mathcal{B}$ .
18:  Generate state value sequence  $(V_{\hat{o}_1}, \dots, V_{\hat{o}_n})$  with MLP layer of Encoder and compute the joint advantage function  $\hat{A}$  based on  $V_{\hat{o}}(\hat{o})$  with GAE.
19:  Input  $(\hat{o}_1, \dots, \hat{o}_n)$  and  $(a_0, \dots, a_{n-1})$ , generate  $(\pi_1^\theta, \dots, \pi_n^\theta)$  with Decoder.
20:  Calculate the training loss  $L = L_{\text{Encoder}}(\phi) + L_{\text{Decoder}}(\theta)$  with Equation 5 and Equation 6 and update Encoder and Decoder.
21: end for
```

4 Experiment

We consider two most common benchmarks: StarCraftII Multi-Agent Challenge [36] and Multi-Agent Particle Environment[25] for evaluating MARL algorithms. Notably, in some domains, our objective goes beyond maximizing win rates or cumulative rewards. We also focus on minimizing the average steps required per episode, highlighting the importance of achieving goals as efficiently as possible.

4.1 Experimental Setting

StarCraftII Multi-Agent Challenge(SMAC). The experiment comprises a set of StarCraft maps where a team of allied units aims to defeat their opponents shown in Fig. 5. Previous studies have shown that HAPPO[37] and MAPPO[38] achieve state-of-the-art performance on this benchmark. By employing parameter sharing, these methods reach near-perfect win rates within most maps, including those featuring heterogeneous agents. Consequently, we hypothesize that non-parameter-sharing approaches are unnecessary, as parameter-sharing techniques sufficiently address the challenges posed by SMAC tasks. To evaluate our work, we test it on four easy, hard and super-hard maps, respectively.

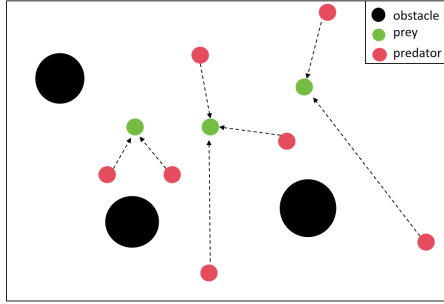


(a) Training map MMM

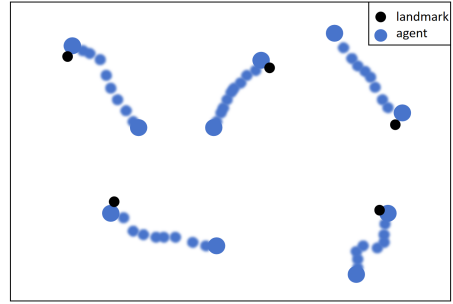


(b) Training map MMM2

Fig. 3: These are two classical maps of StarCraftII Multi-Agent Challenge.



(a) Predator Prey



(b) Cooperative Navigation

Fig. 4: These are two classical scenarios of Multi-Agent Particle Environment.

Multi-Agent Particle Environment (MPE). This experiment utilizes two standard scenarios from the MPE [39]: Predator-Prey and Cooperative Navigation.

Predator-Prey (PP) is a cooperative MARL benchmark task where agents (predators) aim to capture moving preys. In this scenario, N predators work together to catch M preys, as illustrated in Fig. 5a. Each predator has limited access to environmental information and can only interact with other predators within its observation range. Preys move freely within the activity area, actively evading the closest predator. Notably, preys move faster than predators, making individual hunting ineffective. Obstacles add complexity to the chase. The team score for predators is calculated as the negative sum of distances between preys and their nearest predators. Collisions with other predators or obstacles result in a penalty of -1. Each experimental run spans 20 time steps, with the number of agents and enemies set to 3 and 1, respectively.

Cooperative Navigation(CN) represents a classic scenario in the multi-agent particle environment. Agents and landmarks are initialized at random positions at the start of each episode. The objective is for agents to collaboratively occupy all landmarks by adjusting their speeds and directions. Available actions include [up, down, left, right, stop]. Each agent observes its speed, position, and relative displacement with respect to other agents and landmarks. The shared reward is defined as the negative sum of

the distances between each landmark and its nearest agent. Additionally, collisions between agents result in a penalty of -1 to the shared reward. Each episode lasts 20 time steps, requiring agents to learn efficient and accurate strategies for cooperative navigation. Fig. 5b depicts a typical cooperative navigation setup with $N = 5$.

4.2 Hyperparameter design

In our experiments, we ensured consistency in the implementation of baseline methods by utilizing their official repositories. All hyperparameters were kept unchanged from their original configurations to maintain optimal performance. The specific hyperparameters used for different algorithms and tasks are detailed in Tab. 1 and Tab. 3

Table 1: Common hyperparameters

Hyperparameter	Value	Hyperparameter	Value
Critic learning rate	5×10^{-4}	Actor learning rate	5×10^{-4}
Entropy coefficient	0.01	Gain	0.01
Optimizer	Adam	Optimization epsilon	1×10^{-5}
Hidden layer dimension	64	Gradient norm clipping	10
Batch size	3200	Number of mini-batches	1
Rollout threads	32	Training threads	16
Episode length	100	Use GAE	True
Use Huber loss	True		

Table 2: Specific hyperparameters used for MPE in the experiments.

PP	Value	CN	Value
Number Agents	3,15,30	Actor learning rate	5×10^{-4}
Number Enemies	1,5,10	Gain	0.01
episode length	20	Optimization epsilon	1×10^{-5}
Hidden layer dimension	64	Gradient norm clipping	10
Batch size	3200	Number of mini-batches	1
Rollout threads	32	Training threads	16
Episode length	100	Use GAE	True
Use Huber loss	True		

4.3 Baselines

Our team compares our algorithm with powerful baselines without communication and popular communication methods to demonstrate its effectiveness. In the SMAC environment, the baseline methods are based on the CTDE paradigm to ensure a fair comparison:

Table 3: Special hyperparameters in different tasks.

Tasks	PPO Epochs	PPO Clip	Num Blocks	Num Heads	Stacked Frames	Steps	γ
3m	15	0.2	1	1	1	5×10^5	0.99
8m	15	0.2	1	1	1	1×10^6	0.99
1c3s5z	10	0.2	1	1	1	2×10^6	0.99
MMM	15	0.2	1	1	1	2×10^6	0.99
2c vs 64zg	10	0.05	1	1	1	5×10^6	0.99
3s vs 5z	15	0.05	1	1	1	5×10^6	0.99
3s5z	10	0.05	1	1	4	5×10^6	0.99
5m vs 6m	10	0.05	1	1	1	3×10^6	0.99
27m vs 30m	5	0.2	1	1	1	2×10^6	0.99
MMM2	10	0.05	1	1	1	1×10^7	0.99
6h vs 8z	15	0.05	1	1	1	1×10^7	0.99
3s5z vs 3s6z	5	0.05	1	1	1	2×10^7	0.99
PP	10	0.05	1	1	1	1×10^7	0.99
CN	10	0.05	1	1	1	1×10^7	0.99

- **MAPPO**[38] is an extension of the Proximal Policy Optimization (PPO)[35] algorithm tailored for multi-agent system. It uses a clipped surrogate objective to stabilize policy updates, ensuring efficient and reliable learning. In the multi-agent context, MAPPO leverages parameter sharing across agents to improve sample efficiency while maintaining scalability. It operates within the CTDE paradigm, utilizing global state information during training to optimize decentralized policies for execution. This makes MAPPO particularly effective for tasks requiring coordinated behavior in complex environments.
- **HAPPO**[37] is a multi-agent policy gradient algorithm designed within the CTDE framework. It extends the PPO algorithm by introducing decentralized and agent-specific policy updates, ensuring stability without requiring explicit constraints. Unlike traditional hierarchical methods, HAPPO does not explicitly decompose tasks into hierarchical levels; instead, it focuses on efficiently coordinating agents' behaviors in complex environments. This design enables HAPPO to handle diverse and large-scale multi-agent tasks by ensuring robust and scalable policy learning.
- **QMIX**[40] is a value decomposition method designed for multi-agent reinforcement learning. It operates within the CTDE paradigm by learning a joint action-value function that is decomposed into agent-specific Q-values through a mixing network. This mixing network is monotonic, ensuring that the global Q-value can be optimized in a centralized manner while enabling decentralized decision-making during execution. Although QMIX effectively balances individual agent optimization and global coordination, its linear additivity assumption in value decomposition can limit its performance on tasks requiring highly complex interactions among agents.

In the MPE scenarios, we benchmark our approach against various state-of-the-art communication-based MARL baselines. Below are the communication-based methods included in our evaluation:

- **CommNet**[41] implements a shared communication layer that enables explicit message passing among agents at each decision step. It aggregates the hidden states of all agents to generate a global context vector, which is then fed back to each

agent to aid decision-making. This approach facilitates high levels of cooperation by allowing agents to share global information effectively.

- **I3CNet**[13] enhances traditional communication methods by incorporating a selective mechanism. Using attention mechanisms, agents can focus on relevant information from others, reducing redundancy and improving efficiency. This targeted communication is particularly beneficial in environments with heterogeneous agents or dynamic interactions.
- **TarMAC**[15] introduces a targeted communication architecture where agents generate communication messages along with attention weights, enabling selective interaction with specific agents based on task requirements. This point-to-point communication design improves coordination precision and minimizes unnecessary information exchange.

4.4 Main Results

After a series of experiments, we can draw a conclusion that **LF-Commer** can get excellent performance at most scenarios from Tab.4.

In SMAC experiments, benefiting from the communication enhancement provided by the Leader-Follower architecture, LF-Commer demonstrates remarkable performance across most maps, achieving victory rates that reach human grandmaster-level proficiency. In particular, it significantly surpasses baseline methods such as MAPPO, HAPPO, and QMIX, particularly on challenging hard and hard+ tasks. Beyond its outstanding performance, LF-Commer also exhibits consistently lower standard deviations compared to other baselines, underscoring its stable training and testing dynamics.

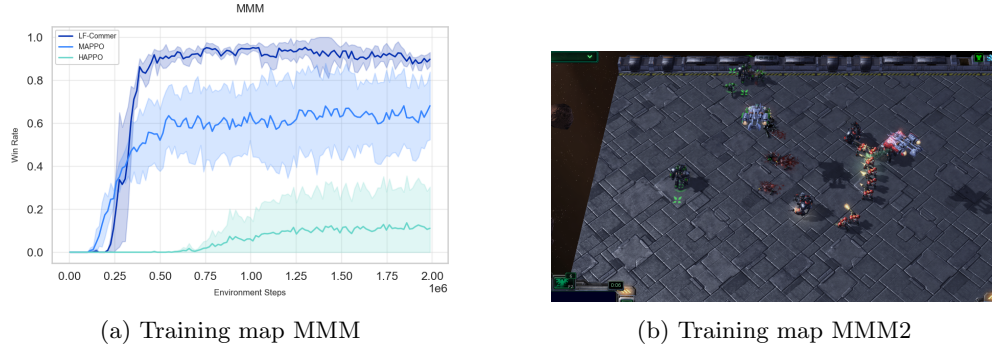


Fig. 5: These are two classical maps of StarCraftII Multi-Agent Challenge.

In small-scale scenarios (e.g., $N=3$), LF-Commer achieves comparable performance to baselines, likely due to the simplicity of coordination. However, as agent count increases ($N=15$), its dynamic leader-follower communication significantly reduces action space complexity, leading to superior cumulative rewards. Specifically, in scenarios involving larger numbers of agents, the mechanism divides the environment into

distinct regions, effectively decomposing complex tasks into manageable local sub-tasks. By leveraging leaders to coordinate the actions of followers, the action space is significantly reduced, allowing superiority of LF-Commer to become increasingly pronounced as the agent grows.

Table 4: Performance Comparison Across SMAC and MPE Scenarios

Task	Difficulty	LF-Commer	MAPPO	HAPPO	QMIX	Steps
3m	Easy	100.0 _(1.8)	100.0 _(0.4)	100.0 _(1.2)	96.9 _(1.3)	5e5
8m	Easy	100.0 _(1.1)	96.8 _(2.9)	97.5 _(1.1)	97.1 _(1.9)	1e6
1c3s5z	Easy	100.0 _(2.4)	100.0 _(2.2)	97.5 _(1.8)	96.9 _(1.5)	2e6
MMM	Easy	100.0 _(2.2)	95.6 _(4.5)	81.2 _(22.9)	91.2 _(3.2)	2e6
2c vs 64zg	Hard	100.0 _(1.3)	100.0 _(2.7)	90.0 _(4.8)	90.3 _(4.0)	5e6
3s vs 5z	Hard	100.0 _(1.7)	100.0 _(2.5)	91.9 _(5.3)	92.3 _(4.4)	5e6
3s5z	Hard	100.0 _(1.9)	72.5 _(26.5)	90.0 _(3.5)	84.3 _(5.4)	5e6
5m vs 6m	Hard	90.6 _(4.4)	88.2 _(6.2)	73.8 _(4.4)	75.8 _(3.7)	1e7
27m vs 30m	Hard+	100.0 _(0.7)	93.1 _(3.2)	0.0 _(0.0)	39.2 _(8.8)	1e7
MMM2	Hard+	93.8 _(2.6)	81.8 _(10.1)	0.3 _(0.4)	88.3 _(2.4)	1e7
6h vs 8z	Hard+	98.8 _(1.3)	88.4 _(5.7)	0.0 _(0.0)	9.7 _(3.1)	1e7
3s5z vs 3s6z	Hard+	96.5 _(1.3)	84.3 _(19.4)	82.8 _(21.2)	68.8 _(21.2)	2e7

Task	Metric	Num	LF-Commer	CommNet	I3CNet	TarMAC
PP	Avg. Cumulative R	3v1	-4.16 _(0.32)	-6.36 _(0.21)	-5.32 _(0.19)	-4.53 _(0.31)
		8v3	-3.84 _(0.21)	-5.73 _(0.22)	-4.86 _(0.35)	-4.75 _(0.42)
		15v7	-2.23 _(0.15)	-8.57 _(1.32)	-9.03 _(0.82)	-7.32 _(0.56)
CN	Avg. Cumulative R	N=5	-0.597 _(0.061)	-0.783 _(0.041)	-0.832 _(0.012)	-0.531 _(0.054)
		N=10	-0.551 _(0.023)	-1.216 _(0.164)	-0.745 _(0.046)	-0.612 _(-0.035)
		N=15	-0.330 _(0.082)	-1.432 _(0.124)	-0.636 _(0.084)	-0.711 _(0.103)

5 Conclusion

In this paper, we propose a novel method named **LF-Commer**, which leverages a leader-follower architecture and multi-agent Transformer to optimize communication and collaboration in multi-agent system. Specifically, we utilize the DBSCAN clustering algorithm to partition agents into regions and designate a leader within each region to gather observation information from its followers. The integrated observation data from the leader-follower framework is then encoded and input into an encoder-decoder structure, which outputs the agents' next actions in an auto-regressive manner. LF-Commer dynamically partitions regions based on task requirements and establishes dedicated leader-follower communication channels. Experimental results demonstrate that this approach achieves significant performance improvements across various cooperative tasks and shows clear advantages over other state-of-the-art baseline methods. Notably, under conditions allowing unrestricted information sharing among all agents, LF-Commer achieves performance close to the theoretical upper limit. This research opens new possibilities for the application of communication learning in the field of

MARL, highlighting the critical role of effective communication in addressing complex challenges.

Acknowledgements

This work was supported in part by the Shenzhen Key Laboratory of Control Theory and Intelligent Systems under Grant ZDSYS20220330161800001 and in part by the Young Scientists Fund of NSFC under Grant 62303209. Recommended by Associate Editor Q1 Vincent Andrieu. (Corresponding author: Xiang Xu.) Chao Wang and Xiang Xu are with Shenzhen Key Laboratory of Control Theory and Intelligent Systems and School of System Design and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: xiangxu5-c@my.cityu.edu.hk).

Declarations

Conflicts of Interest. The authors have no conflict of interest.

References

- [1] Yang, Y., Wang, J.: An overview of multi-agent reinforcement learning from game theoretical perspective. arXiv preprint arXiv:2011.00583 (2020)
- [2] Deng, X., Li, N., Mguni, D., Wang, J., Yang, Y.: On the complexity of computing markov perfect equilibrium in general-sum stochastic games. *National Science Review* **10**(1), 256 (2023)
- [3] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., *et al.*: Human-level control through deep reinforcement learning. *nature* **518**(7540), 529–533 (2015)
- [4] Agrawal, A., Won, S.J., Sharma, T., Deshpande, M., McComb, C.: A multi-agent reinforcement learning framework for intelligent manufacturing with autonomous mobile robots. *Proceedings of the Design Society* **1**, 161–170 (2021) <https://doi.org/10.1017/pds.2021.17>
- [5] Agrawal, A., Won, S.J., Sharma, T., Deshpande, M., McComb, C.: A multi-agent reinforcement learning framework for intelligent manufacturing with autonomous mobile robots. *Proceedings of the Design Society* **1**, 161–170 (2021) <https://doi.org/10.1017/pds.2021.17>
- [6] Zhang, H., Li, J., Qi, Z., Aronsson, A., Bosch, J., Olsson, H.H.: Multi-agent reinforcement learning in dynamic industrial context. In: 2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 448–457 (2023). <https://doi.org/10.1109/COMPSAC57700.2023.00066>
- [7] Zhang, H., Li, J., Qi, Z., Aronsson, A., Bosch, J., Olsson, H.H.: Multi-agent reinforcement learning in dynamic industrial context. In: 2023 IEEE 47th Annual

- Computers, Software, and Applications Conference (COMPSAC), pp. 448–457 (2023). <https://doi.org/10.1109/COMPSAC57700.2023.00066>
- [8] Kraemer, L., Banerjee, B.: Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing* **190**, 82–94 (2016) <https://doi.org/10.1016/J.NEUCOM.2016.01.031>
 - [9] Lo, Y.L., Sengupta, B., Foerster, J.N., Noukhovitch, M.: Learning multi-agent communication with contrastive learning. In: The Twelfth International Conference on Learning Representations (2024). <https://openreview.net/forum?id=vZZ4hhniJU>
 - [10] Lyu, X., Xiao, Y., Daley, B., Amato, C.: Contrasting centralized and decentralized critics in multi-agent reinforcement learning. In: Dignum, F., Lomuscio, A., Endriss, U., Nowé, A. (eds.) *AAMAS '21: 20th International Conference on Autonomous Agents and Multiagent Systems*, Virtual Event, United Kingdom, May 3–7, 2021, pp. 844–852. ACM, ??? (2021). <https://doi.org/10.5555/3463952.3464053> . <https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p844.pdf>
 - [11] Littman, M.L.: Markov games as a framework for multi-agent reinforcement learning. In: *Machine Learning Proceedings 1994*, pp. 157–163. Elsevier, ??? (1994)
 - [12] Hu, S., Shen, L., Zhang, Y., Tao, D.: Learning multi-agent communication from graph modeling perspective. In: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024. OpenReview.net, ??? (2024). <https://openreview.net/forum?id=Qox9rO0kN0>
 - [13] Singh, A., Jain, T., Sukhbaatar, S.: Learning when to communicate at scale in multiagent cooperative and competitive tasks. In: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019. OpenReview.net, ??? (2019). <https://openreview.net/forum?id=rye7knCqK7>
 - [14] Ding, Z., Huang, T., Lu, Z.: Learning individually inferred communication for multi-agent cooperation. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, Virtual* (2020). <https://proceedings.neurips.cc/paper/2020/hash/fb2fcd534b0ff3bbed73cc51df620323-Abstract.html>
 - [15] Das, A., Gervet, T., Romoff, J., Batra, D., Parikh, D., Rabbat, M., Pineau, J.: Tarmac: Targeted multi-agent communication. In: Chaudhuri, K., Salakhutdinov, R. (eds.) *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9–15 June 2019, Long Beach, California, USA. Proceedings of Machine Learning Research*, vol. 97, pp. 1538–1546. PMLR, ??? (2019). <http://proceedings.mlr.press/v97/das19a.html>

- [16] Zhu, C., Dastani, M., Wang, S.: A survey of multi-agent deep reinforcement learning with communication. *Autonomous Agents and Multi-Agent Systems* **38**(1), 4 (2024)
- [17] Wen, M., Kuba, J.G., Lin, R., Zhang, W., Wen, Y., Wang, J., Yang, Y.: Multi-agent reinforcement learning is a sequence modeling problem. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022* (2022). http://papers.nips.cc/paper_files/paper/2022/hash/69413f87e5a34897cd010ca698097d0a-Abstract-Conference.html
- [18] Nadkarni, P.M., Ohno-Machado, L., Chapman, W.W.: Natural language processing: an introduction. *Journal of the American Medical Informatics Association* **18**(5), 544–551 (2011)
- [19] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual* (2020). <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [20] Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (eds.) *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, ??? (2019). <https://doi.org/10.18653/V1/N19-1423> . <https://doi.org/10.18653/v1/n19-1423>
- [21] Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., Jégou, H.: Going deeper with image transformers. In: *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*, pp. 32–42. IEEE, ??? (2021). <https://doi.org/10.1109/ICCV48922.2021.00010> . <https://doi.org/10.1109/ICCV48922.2021.00010>
- [22] Li, K., Lin, K., Zheng, R., Wang, Z.: Selection strategy based on proper pareto optimality in evolutionary multi-objective optimization. In: Affenzeller, M., Winkler, S.M., Kononova, A.V., Trautmann, H., Tušar, T., Machado, P., Bäck, T. (eds.) *Parallel Problem Solving from Nature – PPSN XVIII*, pp. 3–19. Springer,

- [23] Claus, C., Boutilier, C.: The dynamics of reinforcement learning in cooperative multiagent systems. In: Mostow, J., Rich, C. (eds.) *Proceedings of the Fifteenth National Conference on Artificial Intelligence and Tenth Innovative Applications of Artificial Intelligence Conference, AAAI 98, IAAI 98*, July 26-30, 1998, Madison, Wisconsin, USA, pp. 746–752. AAAI Press / The MIT Press, ??? (1998). <http://www.aaai.org/Library/AAAI/1998/aaai98-106.php>
- [24] Littman, M.L.: Markov games as a framework for multi-agent reinforcement learning. In: Cohen, W.W., Hirsh, H. (eds.) *Machine Learning, Proceedings of the Eleventh International Conference*, Rutgers University, New Brunswick, NJ, USA, July 10-13, 1994, pp. 157–163. Morgan Kaufmann, ??? (1994). <https://doi.org/10.1016/B978-1-55860-335-6.50027-1> . <https://doi.org/10.1016/b978-1-55860-335-6.50027-1>
- [25] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., Mordatch, I.: Multi-agent actor-critic for mixed cooperative-competitive environments. In: Guyon, I., Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, December 4-9, 2017, Long Beach, CA, USA, pp. 6379–6390 (2017). <https://proceedings.neurips.cc/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html>
- [26] Foerster, J.N., Farquhar, G., Afouras, T., Nardelli, N., Whiteson, S.: Counterfactual multi-agent policy gradients. In: McIlraith, S.A., Weinberger, K.Q. (eds.) *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), the 30th Innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18)*, New Orleans, Louisiana, USA, February 2-7, 2018, pp. 2974–2982. AAAI Press, ??? (2018). <https://doi.org/10.1609/AAAI.V32i1.11794> . <https://doi.org/10.1609/aaai.v32i1.11794>
- [27] Jiang, J., Lu, Z.: Learning attentional communication for multi-agent cooperation. In: Bengio, S., Wallach, H.M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018*, December 3-8, 2018, Montréal, Canada, pp. 7265–7275 (2018). <https://proceedings.neurips.cc/paper/2018/hash/6a8018b3a00b69c008601b8becae392b-Abstract.html>
- [28] Li, S., Gupta, J.K., Morales, P., Allen, R.E., Kochenderfer, M.J.: Deep implicit coordination graphs for multi-agent reinforcement learning. In: Dignum, F., Lomuscio, A., Endriss, U., Nowé, A. (eds.) *AAMAS '21: 20th International Conference on Autonomous Agents and Multiagent Systems*, Virtual Event, United Kingdom, May 3-7, 2021, pp. 764–772. ACM, ??? (2021). <https://doi.org/>

- 10.5555/3463952.3464044 . <https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p764.pdf>
- [29] Jiang, J., Dun, C., Huang, T., Lu, Z.: Graph convolutional reinforcement learning. In: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020. OpenReview.net, ??? (2020). <https://openreview.net/forum?id=HkxdQkSYDB>
 - [30] Wang, Y., Zhong, F., Xu, J., Wang, Y.: Tom2c: Target-oriented multi-agent communication and cooperation with theory of mind. In: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, ??? (2022). <https://openreview.net/forum?id=2t7CkQXNpuq>
 - [31] Xu, H., Zhu, F., Ling, X.: Secure bipartite consensus of leader-follower multi-agent systems under denial-of-service attacks via observer-based dynamic event-triggered control. *Neurocomputing* **614**, 128817 (2025) <https://doi.org/10.1016/J.NEUCOM.2024.128817>
 - [32] Xiao, J., Wang, Z., He, J., Yuan, G.: A graph neural network based deep reinforcement learning algorithm for multi-agent leader-follower flocking. *Inf. Sci.* **641**, 119074 (2023) <https://doi.org/10.1016/J.INS.2023.119074>
 - [33] Luo, S., Xu, X., Liu, L., Feng, G.: Leader-following consensus of heterogeneous linear multiagent systems with communication time-delays via adaptive distributed observers. *IEEE Transactions on Cybernetics* **52**(12), 13336–13349 (2022) <https://doi.org/10.1109/TCYB.2021.3115124>
 - [34] Ester, M., Kriegel, H., Sander, J., Xu, X.: A density-based algorithm for discovering clusters in large spatial databases with noise. In: Simoudis, E., Han, J., Fayyad, U.M. (eds.) *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, Portland, Oregon, USA, pp. 226–231. AAAI Press, ??? (1996). <http://www.aaai.org/Library/KDD/1996/kdd96-037.php>
 - [35] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017)
 - [36] Samvelyan, M., Rashid, T., Witt, C.S., Farquhar, G., Nardelli, N., Rudner, T.G.J., Hung, C., Torr, P.H.S., Foerster, J.N., Whiteson, S.: The starcraft multi-agent challenge. In: Elkind, E., Veloso, M., Agmon, N., Taylor, M.E. (eds.) *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '19*, Montreal, QC, Canada, May 13-17, 2019, pp. 2186–2188. International Foundation for Autonomous Agents and Multiagent Systems, ??? (2019). <http://dl.acm.org/citation.cfm?id=3332052>
 - [37] Kuba, J.G., Chen, R., Wen, M., Wen, Y., Sun, F., Wang, J., Yang, Y.: Trust

region policy optimisation in multi-agent reinforcement learning. In: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, ??? (2022). <https://openreview.net/forum?id=EcGGFkNTxdJ>

- [38] Yu, C., Velu, A., Vinitzky, E., Gao, J., Wang, Y., Bayen, A.M., Wu, Y.: The surprising effectiveness of PPO in cooperative multi-agent games. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022 (2022). http://papers.nips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html
- [39] Mordatch, I., Abbeel, P.: Emergence of grounded compositional language in multi-agent populations. arXiv preprint arXiv:1703.04908 (2017)
- [40] Rashid, T., Farquhar, G., Peng, B., Whiteson, S.: Weighted QMIX: expanding monotonic value function factorisation for deep multi-agent reinforcement learning. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, Virtual (2020). <https://proceedings.neurips.cc/paper/2020/hash/73a427badebe0e32caa2e1fc7530b7f3-Abstract.html>
- [41] Sukhbaatar, S., Szlam, A., Fergus, R.: Learning multiagent communication with backpropagation. In: Lee, D.D., Sugiyama, M., Luxburg, U., Guyon, I., Garnett, R. (eds.) Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain, pp. 2244–2252 (2016). <https://proceedings.neurips.cc/paper/2016/hash/55b1927fdafef39c48e5b73b5d61ea60-Abstract.html>